# Silver Layer Data Quality Rules

This document defines every data quality expectation applied during the Bronze → Silver transformation pipeline. Each rule is tagged with a severity level and routed to the entity's quarantine table for traceability.

# Architecture Overview

The Silver pipeline is config-driven. All DQ rules are stored in globalmart.config.dq_rules (20 rules across 6 entities) and evaluated at two points:

1. **Inline expectations** (@dp.expect / @dp.expect_or_drop) on the main Silver table — these flag or drop rows in real time during streaming ingestion.
2. **Quarantine engine** — independently re-evaluates every rule against the harmonized data and writes all failures to a paired _quarantine table with full audit metadata: _run_id, _expectation_name, _severity, _issue_type, _failure_reason, and _quarantine_timestamp.

A single row can fail multiple rules and will appear once per failure in the quarantine table.

## Rejection Pattern

| Severity | Decorator | Consequence | When Used |
|---|---|---|---|
| CRITICAL | @dp.expect_or_drop | Row is **dropped** from the main Silver table and captured in quarantine with audit metadata | Missing primary keys or foreign keys — the row cannot participate in any downstream join |
| WARNING | @dp.expect | Row is **kept** in the main Silver table but also captured in quarantine for review | Valid keys but messy data (null emails, bad segments, unparseable dates) — the record is structurally usable but has a flagged quality issue |

---

# Quality Rules by Table

## Customers

Source: globalmart.bronze.customers — 748 rows from 6 regional CSV files.

Schema challenges resolved by the harmonization engine before rules fire:

- 4 different ID column names unified via COALESCE (customer_id, CustomerID, cust_id, customer_identifier)
- customers_5 missing customer_email entirely (42 rows → NULL after harmonization)
- customers_6 uses completely different naming (full_name, email_address, customer_segment, customer_identifier)
- customers_4 has city/state column order swapped in the source header
- Segment abbreviations (CONS, CORP, HO) and typo (Cosumer) mapped to canonical values via CASE WHEN before DQ rules evaluate

| Rule ID | Rule | What Triggers It | Consequence | Severity | Why That Consequence |
|---|---|---|---|---|---|
| CU-001 | customer_id must not be null | COALESCE(customer_id, customerid, cust_id, customer_identifier) IS NULL | Drop + Quarantine | CRITICAL | A customer with no ID across all 4 regional variants cannot be linked to any order, transaction, or return. The record is unusable for any downstream analysis and must be investigated by the finance team. |

| | | | | | |
|---|---|---|---|---|---|
| CU-002 | customer_email must not be null | COALESCE(customer_email, email_address) IS NULL | Warn + Quarantine | WARNING | Email is important for customer communication but is not a key field. Region 5 has no email column at all (42 rows), and Regions 3 and 6 have partial gaps (20.75% and 11.23% respectively). Dropping these records would lose valid customers. Flagged for the finance team to assess whether email recovery is possible from source systems. |
| CU-003 | customer_name must not be null | COALESCE(customer_name, full_name) IS NULL | Warn + Quarantine | WARNING | Name is needed for reporting and deduplication but does not invalidate the customer record. Flagged for investigation rather than dropped. |

| | | | | | |
|---|---|---|---|---|---|
| CU-004 | segment must be a valid value | COALESCE(segment, customer_segment) NOT IN ('Consumer', 'Corporate', 'Home Office') after applying CASE WHEN mapping for abbreviations and typos | Warn + Quarantine | WARNING | Segment drives business reporting by customer type. Abbreviations (CONS, CORP, HO) and the typo (Cosumer) are mapped before this rule fires, so it only catches truly unmappable values. These records remain in the main table since the customer identity is still valid — the segment is flagged for manual correction. |

| | | | | | |
|---|---|---|---|---|---|
| CU-005 | state must pass location swap detection | State column fails regex validation: must be a 2-letter code (e.g. "TX") OR a proper state name (e.g. "Texas"). Triggers when state looks like a city name. | Warn + Quarantine | WARNING | Detects the Region 4 structural anomaly where city and state data are swapped in the source headers. The LLM schema mapper attempts to correct this during harmonization by inspecting sample values, but this rule acts as a safety net. Records are kept because the customer identity is valid — the geography fields are flagged for manual review. |

## Orders

Source: globalmart.bronze.orders — 5,006 rows from 3 regional CSV files.

Schema challenges resolved before rules fire:

- orders_3 missing order_estimated_delivery_date (9 columns vs 10) — Auto Loader adds it as NULL via addNewColumns
- All date columns stored as strings in two competing formats: "MM/dd/yyyy HH:mm" and "yyyy-MM-dd HH:mm" — parsed via COALESCE(TRY_TO_TIMESTAMP()) with both patterns plus a single-digit hour variant

| Rule ID | Rule | What Triggers It | Consequence | Severity | Why That Consequence |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| OR-001 | order_id must not be null | order_id IS NULL | Drop + Quarantine | CRITICAL | Order ID is the primary key and the join key to transactions and returns. Without it, the record cannot participate in revenue calculation, return matching, or any downstream analysis. |
| OR-002 | order_purchase_date must parse to a valid timestamp | COALESCE(TRY_TO_TIMESTAMP(order_purchase_date, 'MM/dd/yyyy HH:mm'), TRY_TO_TIMESTAMP(order_purchase_date, 'yyyy-MM-dd HH:mm'), TRY_TO_TIMESTAMP(order_purchase_date, 'yyyy-MM-dd H:mm')) IS NULL | Warn + Quarantine | WARNING | Purchase date is essential for time-based reporting but the order itself may still be valid. Three date format patterns are attempted across regions — this rule catches records where none parse. Flagged for investigation rather than dropped, since the order may be recoverable with a corrected date. |

| | | | | | |
|---|---|---|---|---|---|
| OR-003 | ship_mode must not be null | ship_mode IS NULL | Warn + Quarantine | WARNING | Ship mode is an analytical dimension, not a key field. orders_2 has an anomalous 11.4% null rate (200 rows) compared to ~1% in orders_1 and 0.8% in orders_3. Dropping would lose valid orders. Flagged so the merchandising team can investigate whether the source system failed to capture this field. |
| OR-004 | order_delivered_customer_date must not be null | order_delivered_customer_timestamp IS NULL | Warn + Quarantine | WARNING | Expected to be null for orders that are not yet delivered (status: shipped, processing, created — together ~2.5% of orders). Logged for monitoring — a rising null rate could indicate a delivery tracking issue. |

| | | | | | |
|---|---|---|---|---|---|
| OR-005 | order_delivered_carrier_date must not be null | order_delivered_carrier_timestamp IS NULL | Warn + Quarantine | WARNING | Same rationale as above. Null is expected for in-transit orders. Logged for trend monitoring. |

## Transactions

Source: globalmart.bronze.transactions — 9,816 rows from 3 regional CSV files.

Schema challenges resolved before rules fire:

- Column casing varies: Order_id vs Order_ID, Product_id vs Product_ID — handled by case-insensitive matching in the harmonization engine
- transactions_1.Sales has '$' prefix on 20.5% of rows (763 of 3,730) — stripped via REGEXP_REPLACE before CAST
- transactions_2 is missing the profit column entirely (3,337 rows → NULL after harmonization)
- payment_type contains '?' placeholders in transactions_1 (357 rows, 9.6%) — mapped to NULL via CASE WHEN

| Rule ID | Rule | What Triggers It | Consequence | Severity | Why That Consequence |
|---|---|---|---|---|---|
| TR-001 | order_id must not be null | order_id IS NULL | Drop + Quarantine | CRITICAL | Foreign key to the orders table. A transaction without an order ID cannot be attributed to any customer, region, or time period. Breaks revenue reporting entirely. |
| TR-002 | product_id must not be null | product_id IS NULL | Drop + Quarantine | CRITICAL | Foreign key to the products table. A transaction without a product ID cannot be attributed to any brand, category, or vendor. Breaks product performance and inventory analysis. |

| | | | | | |
|---|---|---|---|---|---|
| TR-003 | sales must be valid and positive | sales IS NULL OR sales <= 0 after stripping '$' prefix and casting to DECIMAL(10,3) | Warn + Quarantine | WARNING | Sales is the core revenue measure. A null after stripping the $ prefix and casting indicates a non-numeric or empty value in the source. Record stays in the main table since the order-product linkage is still valid, but the revenue figure is flagged as unreliable. |
| TR-004 | profit must not be null | profit IS NULL | Warn + Quarantine | WARNING | Profit is missing entirely for all 3,337 rows from transactions_2. Dropping would remove a third of all transactions. Flagged so the finance team knows profit analysis excludes that region's data until the source system is corrected. |
| TR-005 | discount must not be null | discount IS NULL after stripping non-numeric characters and casting to DECIMAL(5,3) | Warn + Quarantine | WARNING | Discount is an analytical field, not a key. Flagged for completeness tracking. |
| TR-006 | payment_type must not be null or placeholder | payment_type IS NULL (after replacing '?' with NULL) | Warn + Quarantine | WARNING | Payment type is an analytical field, not a key. 357 rows from transactions_1 had '?' placeholders (now NULL), and transactions_2/3 have 1–2% native nulls. Flagged for source system investigation. |

| | | | | | |
|---|---|---|---|---|---|
| TR-007 | quantity must not be null | quantity IS NULL after casting to INT | Warn + Quarantine | WARNING | Quantity is needed for inventory and volume analysis. Flagged rather than dropped since the order-product linkage is intact. |
| TR-008 | profit must not be negative | profit < 0 after casting to DECIMAL(10,3) | Warn + Quarantine | WARNING | Negative profit may be legitimate (loss-making sales with heavy discounts) but could also indicate a sign error or data corruption. Flagged for Finance review — records are kept since loss-making transactions are valid business events. |

## Returns

Source: globalmart.bronze.returns — 800 rows from 2 regional JSON files.

Schema challenges resolved before rules fire:

- Every single column name differs between returns_1 and returns_2: order_id/OrderId, refund_amount/amount, return_date/date_of_return, return_reason/reason, return_status/status — each pair mapped to the same canonical column via COALESCE
- returns_1.refund_amount has '$' prefix on 16.6% of rows (83 of 500) — stripped via REGEXP_REPLACE before CAST
- returns_1.return_status contains abbreviations: PENDG (24 rows), APPRVD (21), RJCTD (20) — expanded via CASE WHEN
- returns_2.reason has '?' placeholders (35 rows, 11.7%) — mapped to NULL via CASE WHEN

| Rule ID | Rule | What Triggers It | Consequence | Severity | Why That Consequence |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| RE-001 | order_id must not be null | COALESCE(order_id, orderid) IS NULL | Drop + Quarantine | CRITICAL | Foreign key linking the return to its original order. Without it, the return cannot be matched to a customer, transaction, or product — making fraud detection and refund reconciliation impossible. |
| RE-002 | return_status must be a valid value | COALESCE(return_status, status) NOT IN ('Pending', 'Approved', 'Rejected') after applying CASE WHEN mapping for abbreviations (PENDG→Pending, APPRVD→Approved, RJCTD→Rejected) | Warn + Quarantine | WARNING | Return status drives the returns processing workflow. Abbreviations are mapped before this rule fires. Any remaining unmappable value is flagged — the return record itself is still valid for fraud pattern analysis even without a clean status. |

| | | | | | |
|---|---|---|---|---|---|
| RE-003 | return_reason must not be null or placeholder | COALESCE(return_reason, reason) IS NULL (after replacing '?' with NULL) | Warn + Quarantine | WARNING | Return reason is needed for root cause analysis (defective products, vendor quality issues). 35 rows from returns_2 had '?' placeholders. Flagged for the returns team — the return is still valid for financial reconciliation even without a reason. |
| RE-004 | refund_amount must be valid and positive | refund_amount IS NULL OR refund_amount <= 0 after stripping '$', removing non-numeric characters, and casting to DECIMAL(10,2) | Warn + Quarantine | WARNING | Refund amount is needed for financial reporting. A null after cleaning indicates a non-numeric source value. Flagged rather than dropped — the return event itself is still relevant for fraud pattern detection. |

# Products

Source: globalmart.bronze.products — 1,774 rows from 1 JSON file.

Schema challenges resolved before rules fire:

- High sparsity preserved as-is: weight (94.36% null), dimension (87.6%), sizes (65.56%), manufacturer (59.02%), upc (39.01%)
- dateAdded and dateUpdated are ISO 8601 strings ("2016-04-01T20:22:54Z") — parsed via TRY_TO_TIMESTAMP()
- product_photos_qty is the only non-STRING column in the entire Bronze layer (LONG type) — passes through without transformation

| Rule ID | Rule | What Triggers It | Consequence | Severity | Why That Consequence |
|---|---|---|---|---|---|
| PR-001 | product_id must not be null | product_id IS NULL | Drop + Quarantine | CRITICAL | Primary key and join key to transactions. A product without an ID cannot be linked to any sale, return, or vendor analysis. |
| PR-002 | product_name must not be null | product_name IS NULL | Warn + Quarantine | WARNING | Product name is needed for reporting and Genie natural language queries. Without a name, the product still participates in joins via product_id but is not human-identifiable. Flagged rather than dropped. |
| PR-003 | brand must not be null | brand IS NULL | Warn + Quarantine | WARNING | Brand is an analytical dimension for product performance reporting. Flagged for completeness tracking. |

## Vendors

Source: globalmart.bronze.vendors — 7 rows from 1 CSV file.

Note: Only 4 of 7 vendors (VEN01–VEN04) are referenced in orders. All 7 retained for referential completeness.

| Rule ID | Rule | What Triggers It | Consequence | Severity | Why That Consequence |
|---|---|---|---|---|---|
| VE-001 | vendor_id must not be null | vendor_id IS NULL | Drop + Quarantine | CRITICAL | Primary key and join key from orders. A vendor without an ID breaks the entire vendor performance and return-rate analysis chain. |

| | | | | | |
|---|---|---|---|---|---|
| VE-002 | vendor_name must not be null | vendor_name IS NULL | Warn + Quarantine | WARNING | Vendor name is needed for human-readable reports and Genie queries. The vendor record is still joinable via vendor_id without a name. Flagged rather than dropped. |

---

# Rule Summary

| Entity | Total Rules | CRITICAL (Drop) | WARNING (Keep) |
|---|---|---|---|
| Customers | 5 | 1 (CU-001) | 4 (CU-002 to CU-005) |
| Orders | 5 | 1 (OR-001) | 4 (OR-002 to OR-005) |
| Transactions | 8 | 2 (TR-001, TR-002) | 6 (TR-003 to TR-008) |
| Returns | 4 | 1 (RE-001) | 3 (RE-002 to RE-004) |
| Products | 3 | 1 (PR-001) | 2 (PR-002, PR-003) |
| Vendors | 2 | 1 (VE-001) | 1 (VE-002) |
| **Total** | **27** | **7** | **20** |

Note: The 27 total includes the 20 rules from globalmart.config.dq_rules plus 7 inline-only expectations (@dp.expect) that are evaluated by the SDP framework but not duplicated in the config table (e.g. valid_quantity, valid_discount, valid_profit_value for transactions).

---

# How Databricks AI/BI Genie Helped Identify Data Issues

During the Bronze table review, we used Databricks AI/BI Genie to query the Bronze tables in natural language and surface data quality issues without writing SQL manually. Specifically:

- **Segment value discovery:** We asked Genie "What are all the distinct customer segment values?" across the unified Bronze customers table. Genie returned 7 distinct values instead of the expected 3, immediately surfacing the abbreviations (CONS, CORP, HO) and the typo (Cosumer). These were not documented in the source data dictionary and would have required manual SELECT DISTINCT queries across multiple columns (segment and customer_segment) to find without Genie.

- **Null pattern analysis:** We asked Genie "Which columns have the most missing values in the transactions table?" and "What percentage of orders are missing a ship mode?". Genie identified the anomalous 11.4% null rate in orders_2's ship_mode compared to ~1% in the other files, highlighting it as a region-specific data capture issue rather than a systemic problem.

- **Format inconsistency detection:** We asked Genie "Show me sample order purchase dates from each source file" which revealed the two competing date formats (MM/DD/YYYY vs YYYY-MM-DD) across regional files — a discrepancy that would have been easy to overlook when reviewing 5,006 order records manually.

Genie's natural language interface allowed us to perform exploratory profiling faster than writing individual SQL queries, and its ability to query across the full Bronze table (with all regional files merged) meant we were profiling the unified dataset rather than checking files in isolation.